

AgentRaft: Raft-Inspired Byzantine Fault Tolerance for LLM Agent Swarms

Dogukan Gundogan

Somnia Agentathon — Encode Club, 2026

Companion code: github.com/DogukanGun/maestro · Paper record: zenodo.org/records/20017799

ABSTRACT

Multi-agent systems built on large language models (LLMs) face a fundamental reliability problem: a single agent can hallucinate — producing confidently wrong output that looks identical to a correct one. When agents are chained naively, one hallucination propagates unchecked through the entire pipeline. We introduce *AgentRaft*, a fault-tolerant consensus protocol for LLM agent swarms that treats hallucination as a Byzantine fault and borrows the structural skeleton of the Raft consensus algorithm to contain it. An elected leader proposes a decision; a quorum of role-diverse followers must independently approve before execution; a supermajority of rejections deposes the leader and rotates to a successor. The entire decision trail is logged in a sealable, content-addressed ledger whose root hash is anchored to Somnia's Shannon EVM testnet via an on-chain event. We describe the protocol, its threat model, its implementation as an open-source TypeScript framework, and its deployment on Somnia. The framework-agnostic core has no Web3 dependency and can be instantiated with any LLM backend.

I. INTRODUCTION

The dominant architecture for complex AI tasks is the *multi-agent pipeline*: a sequence of LLM calls, each consuming the output of the previous one. LangChain, AutoGen, CrewAI, and similar frameworks make such pipelines easy to assemble. What they do not provide is a *correctness guarantee*.

The failure mode is well-known to practitioners: an LLM produces a plausible but wrong intermediate result; downstream agents accept it without challenge; the final output is wrong in a way that is difficult to detect and trace. This is not a crash — the system runs to completion and returns an answer. The answer is just bad.

This failure class is a *Byzantine fault* in the sense of Lamport, Shostak, and Pease [1]: the faulty component does not fail silently, it sends an arbitrary (and potentially misleading) message. Byzantine fault-tolerant (BFT) distributed systems have solved this problem for replicated state machines since Castro and Liskov's PBFT [2], and Raft [3] simplified the crash-fault subset of the same problem. Neither protocol was designed for LLMs. AgentRaft maps them.

Our contributions are:

- A threat model that formally places LLM hallucination within the Byzantine fault class.
- A consensus protocol — *AgentRaft* — that provides safety and liveness under the loyal-majority assumption for a role-diverse follower panel.
- An open-source TypeScript framework implementing the protocol with pluggable port interfaces and in-memory defaults.
- A production-quality adapter layer on Somnia's Shannon EVM testnet, with live deployed contracts, that makes every consensus action a verifiable on-chain transaction and anchors the full log to the chain.

II. BACKGROUND

A. The Byzantine Generals Problem

Lamport, Shostak, and Pease [1] formalized the problem of reaching agreement in a distributed system where some participants may behave arbitrarily — sending conflicting messages to different recipients, lying about their state, or colluding with other faulty nodes. Their central result: reliable agreement requires that the number of traitors t satisfy $t < n/3$ where n is the total participant count. For $n = 4$ (one leader, three followers), tolerating one Byzantine agent is exactly achievable — a fact AgentRaft exploits.

B. Raft Consensus

Raft [3] is a crash-fault consensus protocol designed for understandability. Its key mechanisms — a single elected leader per term, log replication to followers, majority-vote quorum, and leader rotation on failure — are directly reflected in AgentRaft's design. The analogy is structural: Raft terms map to AgentRaft epochs; Raft log entries map to AgentRaft proposals; Raft majority vote maps to AgentRaft's two-thirds quorum; Raft leader failover maps to AgentRaft's deposition.

C. LLM Hallucination as Byzantine Fault

A hallucinating LLM does not crash. It returns a well-formed, confidently stated, syntactically correct response that is semantically wrong. From the perspective of a downstream consumer that cannot independently verify the claim, this is indistinguishable from a correct response — exactly the signature of a Byzantine fault.

The key difference from classical BFT is the *correlation* structure of failures. In a replicated database, Byzantine nodes are adversarial and may coordinate. In an LLM swarm, hallucinations are stochastic: a shared training distribution and a shared context window can cause multiple agents to produce the same wrong answer. Naive majority vote fails under correlated hallucination. AgentRaft's response is *role diversity*: followers are not replicas of the leader but deliberately different epistemic perspectives — Critic, Strategist, Analyst — so that the conditions under which each would independently hallucinate the same wrong approval are disjoint.

III. THE AGENTRAFT PROTOCOL

A. Agents and Roles

An agent in AgentRaft is any object implementing the `Agent` interface (Listing 1). The interface is intentionally minimal: only the `vote` method is required. Optional capabilities — `nominate`, `greet`, `propose`, `summarize` — are taken on by agents that can serve as leader candidates.

```
interface Agent {
  id: string;
  role: 'leader' | 'follower';
  vote(proposal, ctx): Promise<VotePayload>;
  nominate?(ctx): Promise<NominationResult>;
  greet?(ctx): Promise<{ text: string }>;
  propose?(ctx): Promise<{ payload: ProposalPa
  summarize?(ctx, outcome): Promise<{ text: strin
}
```

Listing 1. The Agent interface (TypeScript). Only `vote` is required.

B. Message Types and Wire Format

Every agent action produces a typed `Message` broadcast to the shared `MessageBus`. All message types are Zod-validated at the protocol boundary:

Table 1 — AgentRaft Message Types

TYPE	SENDER	PAYLOAD
NOMINATION	Any agent	nominee id, reasoning
CHAT	Leader	Free-form text
PROPOSAL	Leader	ProposalPayload + actionHash
VOTE	Follower	APPROVE / REJECT + reason
VOTE_NEW_LEADER	Follower	Deposition trigger

The universal message envelope carries `agent_id`, `epoch`, `nonce`, `timestamp`, and an optional `sig` slot for future signature enforcement.

C. Leader Election

On the first invocation, the protocol runs an election at epoch 0. Each agent calls `nominate(ctx)`, an LLM-backed function that returns a `{ nominee, reasoning }` pair via structured output (OpenAI tool-calling). The protocol tallies nominations by simple plurality; ties break by agent list order. The winner is committed via `consensusGate.setLeader()` and persisted to a state file so subsequent invocations skip re-election entirely.

Robustness against LLM nominee hallucination is handled by a cascading normalization pass: exact ID match → lowercased match → substring match → self-nomination fallback.

D. Proposal and Two-Thirds Quorum

At each epoch the current leader calls `propose(ctx)`, producing a `ProposalPayload` containing a human-readable summary, an action object, and a content-addressed `actionHash = SHA-256(JSON.stringify(action))`. The proposal is broadcast and the epoch's log is sealed.

All followers then call `vote(proposal, ctx)` concurrently. Each returns a `VotePayload` with `decision: 'APPROVE' | 'REJECT'` and a one-sentence reason. The quorum rule is `TwoThirdsQuorum`:

```
minApprovals(n) = Math.ceil(2 * n / 3)
```

For three followers this requires two approvals. The same threshold governs the rejection path: two or more rejections trigger deposition.

E. Leader Deposition and Rotation

When the rejection threshold is met, each rejecting follower sends a `VOTE_NEW_LEADER` message and calls `consensusGate.triggerLeaderElection()`. The gate marks the active proposal `blocked` and advances the leader index by round-robin: `leaderIdx = (leaderIdx + 1) % agents.length`. The epoch increments and the new leader begins a fresh proposal cycle. The new leader is aware (via `ctx.history`) of the prior proposal and its rejection reasons, enabling an informed revision.

F. Log Integrity and Content Addressing

At each epoch boundary, `InMemoryLogStore.seal(epoch)` serializes the in-memory message buffer as a JSONL byte string and computes:

```
ref = '0x' + sha256(JSONL_bytes).hex
```

This `ref` is stored in the proposal state and returned in the `SwarmOutcome`. Any consumer can re-derive and verify the log by reading the on-disk `messages.jsonl` file. The `actionHash` is independently computed, so both the deliberation record and the decided action are content-addressed and tamper-evident.

The terminal outcome type makes the content addresses explicit:

```
type SwarmOutcome =
  | { status: 'executed'; epoch: number;
    actionHash: string; logRef: string }
  | { status: 'halted'; epoch: number;
    reason: 'max_epochs_exhausted'
    | 'no_leader_can_propose' }
  | { status: 'greeted'; epoch: number;
    leader: string }
```

Listing 2. `SwarmOutcome` — the terminal result of `runSwarm()`.

IV. PROTOCOL STATE MACHINE

Figure 1 depicts the `runSwarm()` control flow. The election phase (epoch 0) runs only on the first invocation; all subsequent invocations enter directly at the proposal phase with the persisted leader.

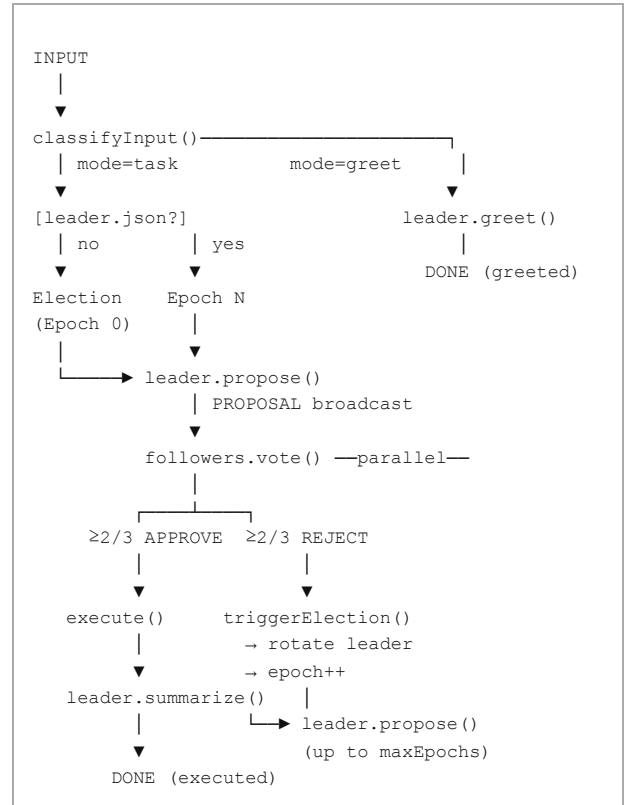


Figure 1. *AgentRaft* protocol state machine. Deposition loops back to the proposal phase with a new leader.

V. SOMNIA CHAIN INTEGRATION

A. Why Somnia

On-chain consensus actions — propose, vote, depose, execute — must be cheap and fast enough not to introduce latency that exceeds user tolerance for an interactive decision tool. Somnia's Shannon testnet (chainId 50312) provides sub-second finality on an EVM-compatible L1, allowing all four consensus actions to complete in the time a human would naturally take to read a proposal. The EVM compatibility means the same Solidity contracts used for testing against a local Hardhat network deploy unchanged to Shannon.

B. *SomniaChainConsensusGate*

The `SomniaChainConsensusGate` class implements the `ConsensusGate` port by translating each protocol operation into a transaction on the deployed `SwarmConsensus.sol` contract:

- `propose(actionHash, logRef)` → `SwarmConsensus.propose()`, emits `Proposed`
- `voteApprove(actionHash, voter)` → `SwarmConsensus.vote(id, true)`, emits `Voted`
- `triggerLeaderElection()` → `SwarmConsensus.triggerElection()`, emits `LeaderDeposed`

- `execute(actionHash)` → `SwarmConsensus.execute()`, emits `Executed`

The gate is initialized with an ethers `Signer` per agent wallet, derived from the swarm mnemonic in the demo configuration.

C. SomniaEventLogStore

Rather than attempting to store full message payloads on-chain (impractical at any block-size budget), `SomniaEventLogStore` implements a "chain-anchored, locally-archived" design: the full JSONL body is written to the local filesystem at `runs/<runId>/messages.jsonl`, and only the keccak256 root hash is committed on-chain via `SwarmConsensus.sealLogBatch(epoch, rootHash)`, which emits `LogBatchSealed(epoch, ref)`.

This gives a tamper-evident guarantee: any modification to the local JSONL file changes its keccak256 hash, which no longer matches the immutable on-chain ref. Integrity is verifiable without storing any content on-chain.

D. SomniaChainIdentityProvider

The `AgentRegistry.sol` contract maintains a stake-gated registry of agent addresses tagged with role strings. `SomniaChainIdentityProvider` reads this registry to resolve agent identities. In the demo deployment, each agent wallet registers with a `0.001 STT` stake — a minimal economic commitment that could be extended to a slashing mechanism in future work.

E. Live Deployed Contracts

Table II — Deployed Contracts, Somnia Shannon Testnet (chainId 50312)

CONTRACT	ADDRESS
AgentRegistry	<code>0xa8420E95A6D43489f6BcD5699E13C2EC7A635d06</code>
SwarmConsensus	<code>0x9476f7Be5A42F2d6f3186eED7f00F1fEf2c18Aa6</code>

Both contracts are verified on the Shannon Explorer at `shannon-explorer.somnia.network`. The full deployment record, including block numbers and transaction hashes, is in `packages/adapters-somnia/deployments.50312.json`.

VI. IMPLEMENTATION

A. Architecture Layers

Table III — AgentRaft Architecture Layers

LAYER	PACKAGE	WEB3
L1 Protocol	<code>core/src/protocol/</code>	No
L2 Ports	<code>core/src/orchestrator/</code>	No
L3 In-memory	<code>core/.../in-memory/</code>	No
L4 Adapters	<code>adapters-somnia/</code>	Yes
L5 Ref app	<code>apps/treasury-demo/</code>	Optional
L6 Dashboard	<code>apps/web/</code>	No

The split is enforced at the package level: `@agentraft/core` declares `zod` as its only runtime dependency, and a CI guard script (`scripts/check-core-deps.mjs`) fails the build if any other runtime dependency is added. This ensures the protocol can be used in any environment — browser, edge function, embedded runtime — without pulling in ethers or any other Web3 library.

B. Reference Application

The `treasury-demo` application provides four agent classes backed by OpenAI's `gpt-4o-mini` (default, overridable). Each class encodes a distinct epistemic role via its system prompt and uses OpenAI tool-calling to produce structured outputs (`ProposalPayload`, `VotePayload`, `NominationResult`) that satisfy the protocol's Zod schemas.

The four roles are deliberately adversarial to correlated hallucination:

- **Coordinator** (leader) — synthesizes and proposes the most actionable solution.
- **Critic** (risk) — searches for risks, blind spots, and faulty assumptions.
- **Strategist** (compliance) — evaluates practical feasibility and long-term constraints.
- **Analyst** (market) — grounds the discussion in data, context, and domain knowledge.

A hallucinated Coordinator proposal must pass all three independent, role-distinct evaluations to be executed. The conditions under which Critic, Strategist, and Analyst would each independently approve the same wrong claim are disjoint by construction.

C. Dashboard

The Next.js 14 dashboard at `apps/web` polls the most recent run directory every 600 ms and live-renders the chat stream, vote tally, and chain events. It exposes three API routes:

- `/api/run` — returns the latest run's JSONL messages and events
- `/api/run/start` — spawns a new `treasury-demo` child process for a submitted task
- `/api/chat` — single-agent quick-ask via the current leader only

VII. EVALUATION

A. Test Coverage

The core protocol is covered by 8 Vitest unit tests covering the message type discriminated union, `TwoThirdsQuorum` and `SimpleMajority` at boundary values, the election tally, the proposal/approve/execute happy path, the deposition rotation, and the max-epoch halting condition. The Somnia adapter contracts are covered by 7 Hardhat tests covering deployment, agent registration with stake enforcement, proposal lifecycle, voting and auto-execute threshold, leader election rotation, and the `sealLogBatch` event emission.

B. End-to-End Verification

An end-to-end onchain proof script (`packages/adapters-somnia/scripts/e2e-onchain.ts`) runs the full `runSwarm()` orchestrator against the real `SwarmConsensus` contract on an in-process Hardhat network. This verifies that the in-memory protocol core and the on-chain adapter are jointly correct — not merely that each passes its own unit tests.

C. Known Limitations

- **No heartbeats.** A timed-out or unresponsive agent will hang `runSwarm()` indefinitely. Crash-fault detection is not implemented.
- **Fixed swarm size.** The framework supports $N \geq 2$ agents; the reference application is fixed at 4. Dynamic membership (join/leave) is not supported.
- **No slashing.** The `AgentRegistry` tracks an `active` flag; deposed leaders forfeit no stake.
- **Demo key management.** Agent wallets are HD-derived from a single mnemonic. Production deployments require per-agent key isolation.

VIII. RELATED WORK

LangChain [4], AutoGen [5], and CrewAI position themselves as multi-agent *orchestration frameworks* — they

provide tools for chaining, routing, and tool-calling across LLMs, but impose no consensus semantics. A hallucination in one step propagates downstream. AgentRaft is not an orchestration framework; it is a *consensus protocol* that can be used within any framework or standalone.

Concurrent work on LLM reliability has explored self-consistency (majority voting over independent samples of the same model) [6] and debate protocols (two models argue for opposing positions before a judge decides) [7]. Self-consistency addresses stochastic sampling variance but fails under correlated hallucination. Debate requires an incorruptible judge. AgentRaft addresses both concerns through role-diverse followers and a quorum gate that has no single point of trust.

On-chain AI coordination has been explored in projects like Galadriel and the Ora Protocol, which focus on bringing LLM inference on-chain. AgentRaft's chain layer is narrower: it anchors *consensus outcomes* on-chain, not inference, keeping costs proportional to the number of decisions rather than the number of tokens.

IX. CONCLUSION

We have presented AgentRaft, a fault-tolerant consensus protocol for LLM agent swarms. By treating hallucination as a Byzantine fault and applying Raft's structural skeleton — elected leader, numbered epochs, quorum voting, leader deposition — we provide a principled mechanism for making multi-agent LLM systems both detectable and recoverable under the loyal-majority assumption.

The framework is available as open-source TypeScript with zero Web3 dependency in the core, a reference implementation backed by OpenAI, a live dashboard, and an adapter layer deployed to Somnia's Shannon testnet. Contracts are live and verifiable at the addresses in Table II.

Future work includes heartbeat-based crash detection, dynamic swarm membership, slashing on deposition, and exploration of alternative quorum rules (e.g., weighted by historical proposal quality). The protocol state machine is amenable to formal verification; we intend to produce a TLA+ specification as a companion artifact.

REFERENCES

1. L. Lamport, R. Shostak, and M. Pease, "The Byzantine Generals Problem," *ACM Transactions on Programming Languages and Systems*, vol. 4, no. 3, pp. 382–401, 1982.

2. M. Castro and B. Liskov, "Practical Byzantine Fault Tolerance," in *Proc. 3rd USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, New Orleans, LA, 1999, pp. 173–186.
3. D. Ongaro and J. Ousterhout, "In Search of an Understandable Consensus Algorithm," in *Proc. 2014 USENIX Annual Technical Conference (ATC)*, Philadelphia, PA, 2014, pp. 305–319.
4. H. Chase, "LangChain," GitHub repository, 2022. [Online]. Available: <https://github.com/langchain-ai/langchain>
5. Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation," *arXiv preprint arXiv:2308.08155*, 2023.
6. X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou, "Self-Consistency Improves Chain of Thought Reasoning in Language Models," in *Proc. 11th International Conference on Learning Representations (ICLR)*, 2023.
7. S. Irving and P. Christiano, "AI Safety via Debate," *arXiv preprint arXiv:1805.00899*, 2018.
8. D. Gundogan, "AgentRaft: Raft-Inspired Byzantine Fault Tolerance for LLM Agent Swarms," Zenodo, 2026. [Online]. Available: <https://zenodo.org/records/20017799>